HW1: Mid-term assignment report

Ana Alexandra Antunes [876543], v2026-10-09

1 Introduction 2
1.1 Overview of the work 2
1.2 Current implementation (faults & extras) 2
1.3 Use of generative AI 2

2 Product specification 2
2.1 Functional scope and actors 2
2.2 Parcel lifecycle 2
2.3 Business rules 3
2.4 Non-functional requirements 3
2.5 System architecture and design decisions 3
2.6 API for developers 4

3 Quality assurance 4
3.1 Overall strategy for testing 4
3.2 Traceability matrix 4
3.3 Unit tests and test doubles 5
3.4 Integration tests 5
3.5 Functional tests (web automation) 5
3.6 Acceptance tests (BDD) 5
3.7 Non-functional testing 5
3.7.1 Performance 5
3.7.2 Accessibility 5
3.8 Code quality analysis 5
3.9 Continuous integration pipeline [optional] 6

4 References & resources 6

1 Introduction

1.1 Overview of the work

This report presents the midterm individual project required for TQS, covering both the software product features and the adopted quality assurance strategy.
The product, named TuttiBox, is a smart parcel locker management platform. The solution provides a Spring Boot REST API and a React web platform that allows online shops (Partners) to register parcels, Couriers to drop them off at locker stations, Recipients to securely pick them up, and Operators to monitor the network.

1.2 My current implementation (limitations & extras)

The current implementation successfully covers the core use cases UC1–UC6.

Limitations: In the web page the responses are not displayed in a pretty way, nor are the errors, the JSON message is simply printed in the page.

Extras implemented: I developed a complete Single Page Application (SPA) frontend using React, TypeScript, and Tailwind CSS. The backend, frontend, and PostgreSQL database are fully containerized using Docker Compose. I implemented a Continuous Integration (CI) pipeline via GitHub Actions that automatically runs the build and enforces a code quality gate using SonarQube Cloud. The core parcel state machine was also extended beyond the minimum requirements to include IN_TRANSIT and DESTROYED states to better track the parcel lifecycle, although those two states do not have much functionality due to time constraints.

1.3 Use of generative AI

Generative AI was used to assist with boilerplate generation for the React/Tailwind frontend components, Spring Boot Docker configuration, general REST API structuring, writing some of the code for the tests, to help in debbuging both configurations and code and finally to help write the report.

The AI assistants used were the Sonnet 4.6 (low) model on claude.ai, gemini on the web page and Big Pickle with opencode.

The quality assurance strategy, including test design and test double selection, was not delegated.

Audit example: Code obtained from the AI was strictly audited. For instance, an initial AI suggestion used a generic advanceState() method that bypassed state machine validation. I identified this flaw and refactored the domain entity to use explicit domain events (e.g., markAsReadyForPickup()) guarded by a centralized isNextStateValid() check to ensure robust business logic and enforce proper timestamped history tracking.

2 Product specification

2.1 Functional scope and actors

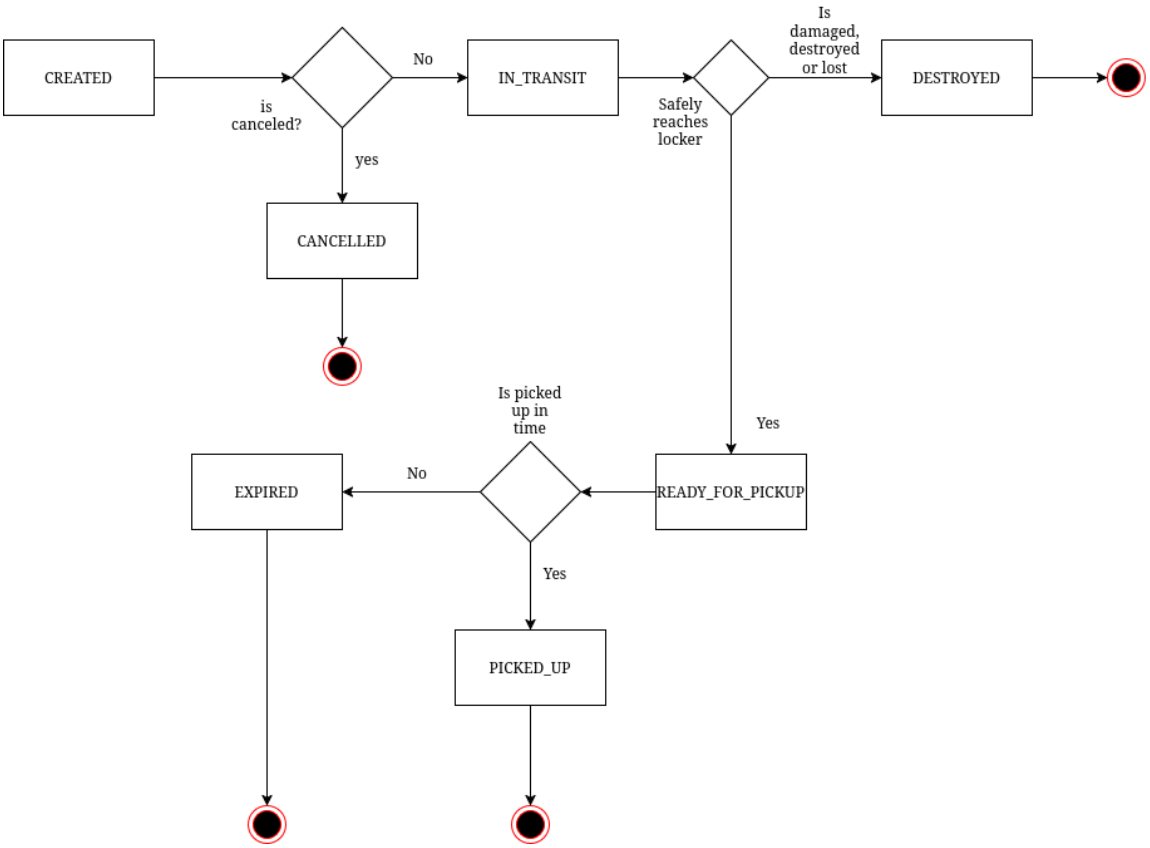
ID	User story	Acceptance criteria	Acceptance test?
US1	*As a recipient, I want to enter my address, so that I can find the nearest locker stations.*	System geolocates address and shows nearest stations with available compartments.	Cucumber / Playwright

ID	User story	Acceptance criteria	Acceptance test?
US2	*As a partner, I want to register a parcel, so that I can reserve a compartment.*	System validates size/contact, reserves a compartment, and returns a tracking code.	Cucumber
US3	*As a courier, I want to drop off a parcel, so that it becomes ready for pickup.*	Tracking code validated, door closed, state updates to READY_FOR_PICKUP, pickup code issued.	Playwright
US4	*As a recipient, I want to pick up a parcel, so that I can retrieve my item.*	Valid pickup code entered, compartment opens, parcel delivered, compartment freed.	Playwright
US5	*As a user, I want to track my parcel, so that I can see its history and current state.*	System returns current state and the timestamped history of transitions.	Cucumber
US6	*As an operator, I want to supervise stations, so that I can manage expired parcels and faults.*	Dashboard lists parcels by state and allows registering the removal of expired parcels.	Playwright

2.2 Parcel lifecycle

The parcel lifecycle utilizes a strict state machine comprising the following states: CREATED, IN_TRANSIT, READY_FOR_PICKUP, PICKED_UP, EXPIRED, RETURNED, CANCELLED, and DESTROYED. Every transition requires an explicit domain event, is validated by the system, and automatically logs the timestamp and actor into a StateHistory embedded collection.

CREATED: First state of the parcel, attributed after creation.
CANCELLED: Attributed to parcel if the recipient cancels it.
IN_TRANSIT: State representing the trip from the “warehouse” or storage to the locker.
DESTROYED: Represents a state where the parcel never got to the locker.
READY_FOR_PICKUP: Attributed when the parcel gets to the locker and the door is closed.
EXPIRED: Attributed if the recipient never got the parcel until the parcel pickup expiry time.
RETURNED: Attributed after an expired parcel is returned to the courier.
PICKED_UP: Final state for when the recipient picks up the parcel and closes the door.



2.3 Business rules

Rule/theme	Value/rationale	Configured in	Tested in
Compartment allocation	Parcels strictly match compatible sizes (S, M, L).	ParcelService	Integration Tests
Drop-off deadline	Time allowed for drop-off evaluated against the createdAt timestamp.	Parcel entity	Unit Tests
Pickup deadline	Time allowed for pickup evaluated against the readyForPickupAt timestamp generated upon drop-off.	Parcel entity	Unit Tests
Pickup code (format, validity, protection against guessing)	Uses a randomly generated UUID to guarantee single-use and protection against guessing.	Parcel entity	Unit Tests
Hardware failure: Door failed to lock (Drop-off)	Sensor timeout on drop-off flags compartment as FAULTY, aborts state transition, and notifies courier.	LockerHardwareFake / ParcelService	BDD (Cucumber) & Unit Tests

Rule/theme	Value/rationale	Configured in	Tested in
Hardware failure: Door failed to open (Pickup)	Unlock failure leaves parcel state as READY_FOR_PICKUP, keeps pickup code valid, and flags compartment for operator inspection.	LockerHardwareFake / ParcelService	Unit & Integration Tests

2.4 Non-functional requirements

ID	Requirement (measurable)	Verification method/tests implemented
NFR1	**Performance:** API response times for tracking a parcel (GET /api/v1/parcels/{trackingCode}) must remain below 200ms at the 95th percentile (p(95) < 200ms) under a load of 20 concurrent virtual users over 30 seconds.	Automated load test using k6 CLI (load/track-parcel.js).
NFR2	**Accessibility:** Frontend interfaces (Find Locker, Track Parcel, Locker Terminal, and Operator Dashboard) must comply with WCAG 2.1 AA standards for keyboard navigation and screen contrast.	Automated scans via Axe-core / browser dev tools.
NFR3	**Security of Pickup Code:** Pickup codes must use 128-bit UUID generation, be single-use, and be mathematically protected against guessing/brute-force attacks.	Unit tests on code entropy and API integration tests asserting code invalidation after single use.
NFR4	**External API Policy Compliance:** Geocoding service client (OpenStreetMap Nominatim / GeoAPI.pt) must enforce strict timeouts and fallbacks, with zero calls to the live API during builds or load tests.	Integration test suite using WireMock HTTP stubs.

2.5 System architecture and design decisions

Design decision	Your option	Testing strategy/impact
Simulation of the locker terminal	React Single Page Application component (LockerTerminal.tsx) interacting with backend REST API endpoints (/dropoff, /pickup), completely decoupled from physical hardware.	Automates end-to-end user flows (courier deposit and recipient collection) using Playwright browser automation without requiring cyber-physical hardware.

Design decision	Your option	Testing strategy/impact
Representation of the parcel lifecycle	Encapsulated state machine inside the Parcel entity, enforced via exhaustive switch expressions and logged to an embedded @ElementCollection	Yields 100% line and branch unit test coverage in JaCoCo by eliminating dead default compiler branches and preventing illegal state transitions.
Controllable time in tests	Injected java.time.Clock bean into ParcelService and domain logic to evaluate createdAt and readyForPickupAt deadlines.	Enables deterministic unit and BDD tests for drop-off and pickup expiry using Clock.fixed() without resorting to Thread.sleep().
Hardware port: operations; synchronous or event-driven	Synchronous interface operations (openCompartment(), verifyDoorClosed()) backed by an in-memory LockerHardwareFake maintaining compartment lock/door states.	Allows direct failure injection (e.g., simulating a stuck door) in unit and BDD tests to verify system rollback and faulty compartment flagging.
Concurrency in compartment allocation	Transactional allocation in ParcelService backed by database pessimistic/optimistic locking during station compartment queries.	Validated via concurrent multi-threaded Spring Boot integration tests asserting that two simultaneous registrations never claim the last compartment.
Pickup codes: generation, storage, checking	Randomly generated 128-bit UUIDs instantiated upon drop-off (READY_FOR_PICKUP), persisted in PostgreSQL, validated for single-use, and cleared upon pickup.	Tested at the unit level for entropy/uniqueness and at the API integration level for single-use enforcement and brute-force/guessing protection.
Geocoding provider: choice; behavior when slow, unavailable or over limits	External REST client (Nominatim / GeoAPI.pt) configured with connection timeouts, retry bounds, and domain exception fallbacks.	ntegration tests utilize WireMock / HTTP Stubs to verify system resilience and explicit error reporting during HTTP 503 errors, rate-limiting, or slow responses.
REST API design: resources, status codes, error format	Standard noun-based URIs (/api/v1/parcels), explicit HTTP verbs (POST, GET, PUT), standard status codes (201 CREATED, 400 BAD_REQUEST, 409 CONFLICT), and structured JSON error DTOs.	Simplifies API contract verification using MockMvc and Cucumber BDD step definitions while preventing mass-assignment security vulnerabilities

2.6 API for developers

The API follows standard RESTful principles using noun-based URIs and appropriate HTTP verbs.

API documentation at http://localhost:8080/swagger-ui/index.html#/ after running *docker compose up –build*

3 Quality assurance

3.1 Overview of the tests available

The testing strategy relies on a standard testing pyramid. Unit tests isolate domain logic and state transitions using mock dependencies. Integration tests use Testcontainers to verify database and repository layers. Finally, BDD and end-to-end (E2E) tests cover the full stack. The tests are executed via Maven (mvn verify). For the E2E (Playwright) tests the docker containers must be running and the –Dbrowser.tests=true must be passed to mvn verify.

3.2 Unit tests and test doubles

Unit tests focus heavily on the Parcel domain entity and the ParcelService. I utilized Mockito to create Mock test doubles for the ParcelRepository to verify behavior without hitting the database. For time-sensitive business rules (deadlines), I utilized a Stub by passing Clock.fixed() to ensure time was fully deterministic during test execution.

As for the two fakes, one of them is the LockerHardwareFake to maintain an in-memory map of compartment states, track door physical lock positions, recognize an open or closed door and keep an operational status without requiring an overly complex Mock stub. The other fake was the Geocoding port fake as to not hit the API and avoid rate limiting penalty blocks by keeping a Map of <String,Coordinates> pre-populated with known addresses and pre-calculated GPS coordinates.

3.3 Integration tests

My integration testing approach verifies the cohesive interaction between the application layer, the database, and external HTTP clients. Core domain rules, such as the parcel state machine transitions, were exhaustively tested at the unit level, achieving perfect branch coverage by utilizing modern Java switch expressions. Time-based rules governing drop-off and pickup deadlines were also unit-tested deterministically by injecting a stubbed fixed clock. To isolate these tests, I implemented two custom fakes. The hardware port fake maintains an in-memory state of compartment doors and sensors, allowing me to inject physical hardware failures like a locked door or a timed-out sensor. The geolocation port fake keeps an in-memory address map and can inject HTTP unavailability, rate limits, and network latency. For service-level unit tests, I used Mockito to mock the persistence layer, enabling behavior verification without database overhead. Finally, I utilized full-context Spring Boot tests alongside Testcontainers to verify the entire request lifecycle against a real PostgreSQL

database. This ensured that the REST API, concurrency locking during compartment allocation, and database mappings functioned together correctly, while the external geocoding client was tested against a stubbed server to prevent live API calls during the build.

3.4 Functional tests (web automation)

My functional testing strategy validates end-to-end user journeys on the frontend React single-page application to ensure the user interface correctly triggers the backend API logic. I used Playwright to automate the core locker terminal flows, specifically the courier drop-off and recipient pick-up actions. I also automated the operator dashboard to verify that operators can list parcels and correctly monitor the network. To ensure these automated tests remained stable and immune to generic layout changes, I used explicit data test attributes as element locators in the frontend code. The tests were designed to rely on fixed staging data containing pre-seeded locker stations, ensuring a consistent execution environment as mandated by the project scope.

3.5 Acceptance tests (BDD)

Acceptance testing translates the business requirements directly into executable specifications using Cucumber and Gherkin syntax. I wrote scenarios covering the core use cases, including a time-based scenario that simulates a parcel registration and evaluates the drop-off expiration deadline. I also included a hardware-failure scenario where a courier attempts a drop-off, but the door sensor times out, successfully verifying that the system handles physical hardware anomalies properly by rejecting the state change. These step definitions drive the system directly at the REST API layer. Driving the API instead of the user interface ensures that business logic, state transitions, and boundary rules are verified rapidly and reliably while avoiding the inherent flakiness of browser automation.

3.6 Non-functional testing

Performance:
Scenario & Requirement: Verified NFR1, ensuring API response times for tracking a parcel (UC5) remain below 200ms under load.
Tool & Setup: A js script using k6 was configured to simulate 20 concurrent users fetching parcel tracking data.

Results:

```
  TOTAL RESULTS

    checks_total.......: 2360    78.243935/s
    checks_succeeded...: 100.00% 2360 out of 2360
    checks_failed......: 0.00%   0 out of 2360

    ✓ status is 200 OK
    ✓ response time < 200ms

    CUSTOM
    error_rate.....................: 0.00%  0 out of 1180

    HTTP
    http_req_duration..............: avg=8.28ms   min=1.95ms  med=7.09ms   max=109.38ms p(90)=12.74ms  p(95)=15.76ms
      { expected_response:true }...: avg=8.28ms   min=1.95ms  med=7.09ms   max=109.38ms p(90)=12.74ms  p(95)=15.76ms
    http_req_failed................: 0.00%  0 out of 1181
    http_reqs......................: 1181   39.155122/s

    EXECUTION
    iteration_duration.............: avg=509.08ms min=502.1ms med=507.93ms max=544.9ms  p(90)=513.69ms p(95)=516.58ms
    iterations.....................: 1180   39.121967/s
    vus............................: 20     min=20        max=20
    vus_max........................: 20     min=20        max=20

    NETWORK
    data_received..................: 685 kB 23 kB/s
    data_sent......................: 143 kB 4.7 kB/s



running (0m30.2s), 00/20 VUs, 1180 complete and 0 interrupted iterations
default ✓ [======================================] 20 VUs  30s
```

Accessibility:

Automated Checks: Axe-core was run against the React SPA (Find Locker, Terminal, and Dashboard pages). 3 problems were found and fixed. The user field in the Track Parcel page had too low contrast and the text color was changed from gray-400 to gray-500. The Pick Up button in the Terminal page had too little contrast, and as such the bg color was changed from green-700 to green-800. Finally in the Operator dashboard the Parcels table was not scrollable with keyboard only and so I added tabIndex={0} to make it possible.

3.7 Code quality analysis

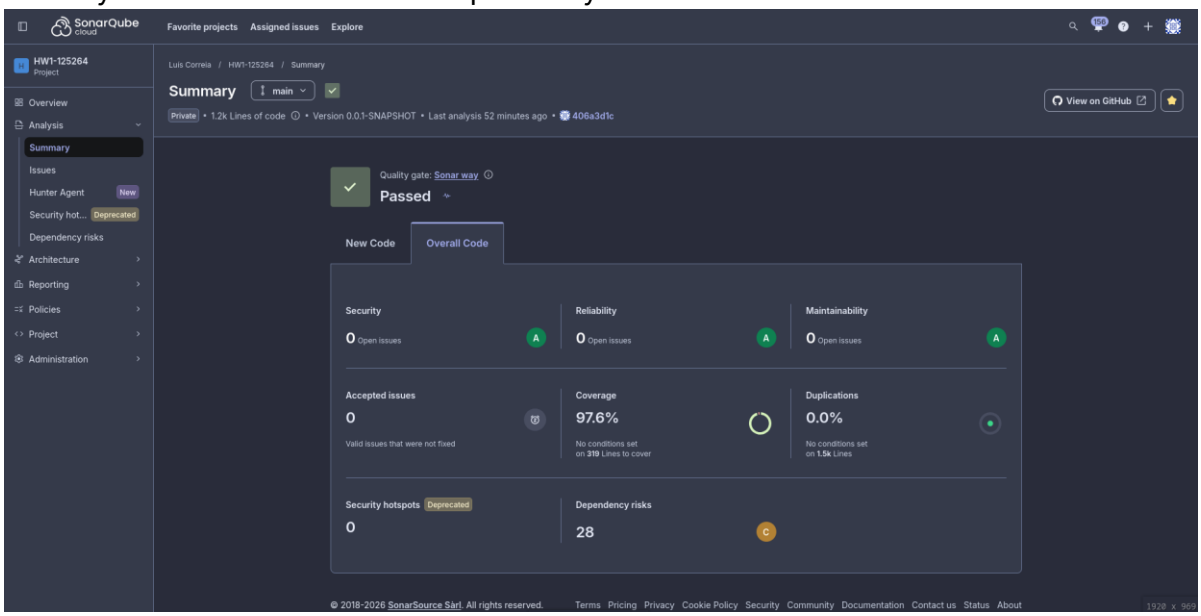
JaCoCo was utilized for static code coverage analysis. The quality gate used and defined was the default sonarqube cloude quality gate.

Lessons learned: Initially, exhaustive enum switch statements generated hidden default compiler branches that lowered coverage. I refactored the code to use modern Java 14+ switch expressions, which are evaluated as exhaustive by the compiler. This allowed the Parcel state machine to achieve a perfect 100% branch and line coverage without requiring dead code testing. Code quality was further enforced by SonarQube Cloud

3.8 Continuous integration pipeline [optional]

I implemented a CI pipeline using GitHub Actions (located in the .github/workflows directory). The pipeline automatically triggers on commits to run the Maven build, execute all test suites, and enforce the Quality Gate via SonarQube Cloud to prevent regressions and maintain code standards.

As seen the test coverage is at 97.6%, way above the defined quality gate, and there are not Security concerns other than the dependency risks.



4 References & resources

Project resources

Resource	URL/location
Git repository	https://github.com/LuisPCNeri/TQS-Portfolio
Video demo	https://github.com/LuisPCNeri/TQS-Portfolio/blob/main/HW1-125264/HW1-125264/docs/simplescreenrecorder-2026-11-05_22.06.16.mp4
API documentation	http://localhost:8080/swagger-ui/index.html#/
QA dashboard (online)	https://sonarcloud.io/project/overview?id=HW1-125264
CI/CD pipeline	https://github.com/LuisPCNeri/TQS-Portfolio/blob/main/.github/workflows/main.yml

Reference materials